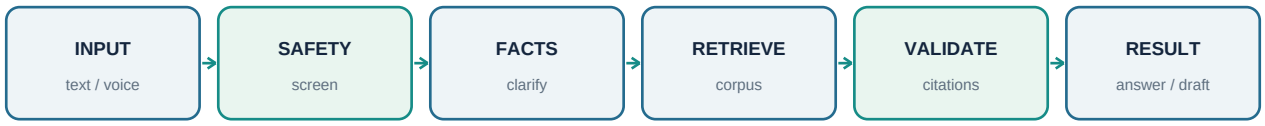
AI LAWYER INDIA

End-to-end project pipeline

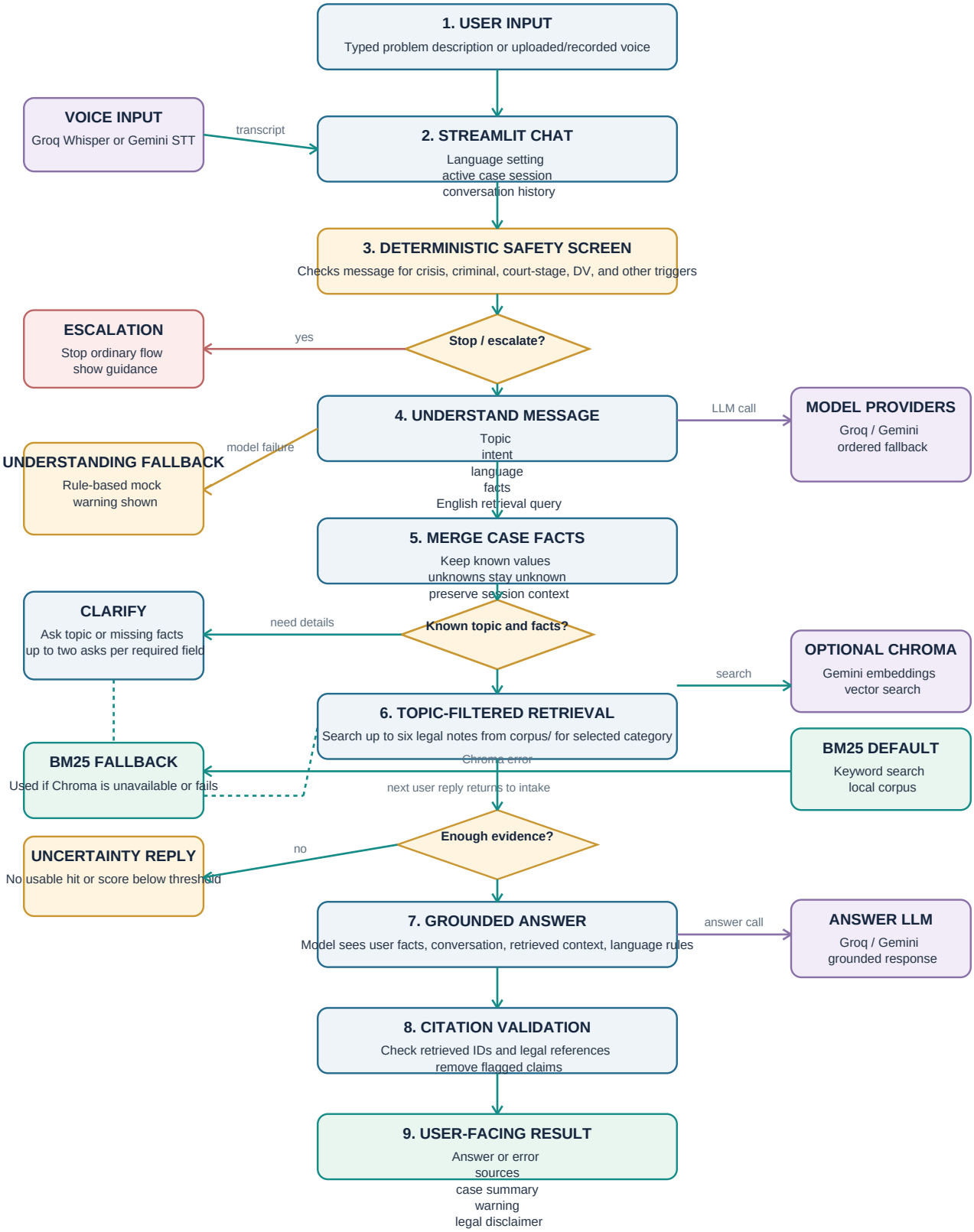
A visual guide to how a message moves through the application, how the system uses the legal knowledge base, and how answers, drafts, voice, and safety handling are connected.



Branches: escalation | follow-up questions | uncertainty | document and practice features

The main path is deliberately conditional: urgent or out-of-scope messages can be redirected, missing facts trigger follow-up questions, weak retrieval triggers uncertainty, and model failures can produce a clear error or a limited understanding fallback.

End-to-end flow and decision branches



Every new chat turn re-enters the session workflow with current case facts and conversation history.

Main branch: safety pass -> understand -> collect needed facts -> retrieve -> check evidence -> answer from context -> validate -> display.

1. Input, session, and safety gates

Stage	What happens	Main implementation
Text input	The user enters a problem or follow-up in the Streamlit chat.	app.py
Voice input	The recorded/uploaded audio is sent to Groq Whisper; Gemini can be attempted as a fallback. The transcript enters the same chat path.	app.py, lawyer/voice.py
Session	A case session keeps facts, asked fields, language, history, retrieval index, and feature state for the conversation.	lawyer/pipeline_v2.py
Language	An explicit language selection takes priority; otherwise the application detects language from the message.	lawyer/language.py
Hard safety check	Deterministic patterns inspect messages before ordinary LLM understanding.	lawyer/escalation.py
Stop/escalate	Stop-level concerns are routed to an escalation response. Other warnings can accompany a general response.	lawyer/escalation.py, pipeline_v2.py

What happens at the safety gate

- The application first checks the user's wording for known high-risk or out-of-scope patterns.
- If a stop-level condition is detected, the regular information-answer path stops and an escalation message is returned.
- If no immediate stop is triggered, the understanding step classifies the issue and can add model-identified escalation categories.
- Messages sent during practice mode follow a restricted path: a crisis signal can interrupt the role-play.
- The checks are rule-based and cannot identify every possible risk or interpretation.

The workflow keeps the UI responsive to errors: if the model cannot be reached for answer generation, the user receives an error response rather than an invented answer.

2. Understanding, fact collection, and retrieval

Understanding and case facts

In real-provider mode, the understanding prompt asks for a structured result: problem category, escalation type, facts, user intent, search query, summary, and language. The conversation context and known facts are supplied to support follow-up turns. In mock mode, rule-based understanding is used instead.

- Supported categories: deposit, consumer, salary, civil, and family information.
- Required fact fields differ by category. Deposit cases can request state, other party, amount, event date, and agreement; other supported categories have their own smaller field sets.
- Missing required fields can be asked about at most twice each before the workflow continues with available information.
- An unrecognized category prompts the user for clarification and can eventually be escalated as out of scope.
- Known session facts are merged without replacing them with empty values; unknown facts are not guessed.

Retrieval branches

Path	Behavior	Decision or fallback
BM25	Tokenizes user query and corpus text, ranks notes using BM25, and filters by selected topic plus general notes.	Default keyword search; local corpus.
ChromaDB	Optional vector retrieval uses Gemini embeddings and a persistent Chroma index.	Selected only when configured.
Chroma initialization/query failure	The retrieval layer falls back to the BM25 index.	Reports fallback in logs and retrieval label.
Retrieval confidence	The answer path checks whether the best match reaches the configured minimum score.	No results or low score returns an uncertainty response.
Context assembly	Up to six retrieved chunks are formatted with identifiers, Act names, optional sections, and note text.	Context is provided to answer generation.

Key retrieval components

lawyer/rag.py parses corpus chunks, excludes quarantined and metadata notes, builds BM25 search, filters by topic, and provides a BM25 fallback. **lawyer/rag_chroma.py** provides optional vector indexing and search. The corpus is stored under **corpus/**.

3. Grounded answer, validation, and response

Answer generation

When retrieval is strong enough, the system sends the selected facts, recent user messages, current question, language requirements, and retrieved notes to the configured model. The response instructions require legal claims to be grounded in the context and cite chunk identifiers. The first substantive answer has a different structure from later follow-up responses.

Check	Implementation behavior
Provider selection	Groq and Gemini are attempted in the configured LLM_PROVIDER order.
Understanding failure	The workflow can use rule-based understanding and mark the interaction as degraded.
Answer failure	If answer generation fails across configured providers, the user receives an explicit service error.
Retrieval failure	Weak or absent evidence returns uncertainty instead of a generated legal explanation.
Citation validation	Citations and references are checked against retrieved notes; unsupported claims can be stripped or flagged.
Final presentation	The UI displays answer, retrieved notes, citation warnings, case summary, audio where available, and the legal-information disclaimer.

What the user can see

- A case summary and evidence checklist can appear with the first substantive answer.
- An expandable sources panel lists each retrieved chunk, verification metadata, score, and note text.
- Warnings can explain escalation or limited state-specific coverage.
- A legal-information disclaimer is shown with the conversation.
- If enabled and supported for the detected language, gTTS can generate spoken output; it is optional and does not change the answer flow.

Citation checks only test references against project materials. They do not prove that a claim is legally correct, complete, or current.

4. Optional features and output paths

Feature	Pipeline	Result and boundary
Document draft	User requests a draft; supported categories ask for confirmation, then the code builds template context from known facts and optional sidebar details.	Jinja2 templates produce DOCX and PDF. Missing fields remain placeholders. Family-court drafting is not supported.
Draft validation	Draft text is checked against relevant corpus notes for legal references.	Validation flags do not replace professional review.
Practice mode	After case intake, role-play uses a supported topic, retrieved context, and practice history.	The other-party role and feedback are constrained by practice prompts and corpus context.
Speech-to-text	Audio is sent to configured Groq Whisper; Gemini may be tried when available.	Transcript is handled as chat input; provider availability applies.
Text-to-speech	The displayed response can be passed to gTTS when the option is enabled and language is supported.	Optional audio output; unsupported languages return no audio.

Document path

- The model does not free-write a legal document; the project uses fixed templates.
- Facts and optional user-provided details are inserted into the selected category template.
- Missing information is marked [TO BE FILLED].
- The generated document and review warning are returned to the UI for download.

Practice path

Practice mode is available for selected deposit, consumer/refund, salary, and civil money matters. It maintains its own short role-play history, can provide feedback, and is interrupted by the crisis safety check. Family matters are directed to a lawyer.

5. Configuration, deployment, and component map

Configuration	Purpose
LLM_PROVIDER	Provider order for language-model calls, such as Groq followed by Gemini.
GROQ_API_KEY / GEMINI_API_KEY	Credentials for configured provider services; must be stored as secrets.
GROQ_MODEL / GEMINI_MODEL	Chat model identifiers used by each provider.
RETRIEVAL_BACKEND	BM25 by default or optional ChromaDB vector retrieval.
GEMINI_EMBEDDING_MODEL	Embedding model used when ChromaDB is enabled.
MIN_RETRIEVAL_SCORE	BM25 minimum score used for answer uncertainty handling.

Deployment and runtime data

The public GitHub repository supplies the source for the Streamlit Community Cloud app. The cloud app runs app.py from the main branch and installs packages from requirements.txt. A local .env file is not automatically deployed. The configuration module reads environment variables; cloud secret values must be made available to that configuration path. Active chat facts are session state, corpus notes are project files, generated drafts are runtime outputs, and Chroma data is optional.

Observed deployment note: the hosted app was reachable, but a live test returned Groq HTTP 404 for llama-3.3-70b-versatile. The app fell back to rule-based understanding. The local configuration showed openai/gpt-oss-120b, so the tested cloud model setting did not match that local value.

Main code map

File or folder	Pipeline role
app.py	Streamlit interface, text/voice controls, actions, and response rendering.
lawyer/pipeline_v2.py	Conversation session, main workflow, retrieval-to-answer path, documents, practice.
lawyer/escalation.py; extraction.py; language.py	Safety signals, required facts and questions, language identification.
lawyer/rag.py; rag_chroma.py; corpus/	Corpus parsing, BM25, optional Chroma retrieval, legal reference notes.
lawyer/llm.py; voice.py; mock.py	Provider calls, speech input/output, and offline fallback behavior.
lawyer/validator.py	Citation and legal-reference checks.
lawyer/documents.py; templates/	Template document generation and export.
lawyer/practice.py	Practice role-play and feedback.
tests/	Automated tests, evaluation cases, and evaluation runner.

6. Complete pipeline summary

#	Pipeline step	Decision / result
1	Receive text or voice input.	Voice is transcribed; text enters the chat session.
2	Load session context and detect language.	Maintain current facts and recent history.
3	Run deterministic safety checks.	Stop/escalate selected risk signals before routine answers.
4	Understand message and classify topic.	LLM JSON understanding or mock/rule-based fallback.
5	Merge known facts and check required fields.	Ask follow-up questions; do not guess missing values.
6	Search legal knowledge base.	BM25 default; optional Chroma; BM25 fallback on Chroma failure.
7	Check retrieval strength.	Weak evidence returns an uncertainty response.
8	Generate grounded answer from retrieved notes.	Provider receives facts, question, language and context.
9	Validate references and citations.	Strip or flag unsupported references before display.
10	Render response and optional features.	Show summary, sources, disclaimer, audio, draft, or practice flow.

Project links

Repository: https://github.com/ayan27mondal/KukuAi_Layer/
Hosted application: https://kukuailayer-d9uc5ewttlsucxor5bcctq.streamlit.app/

Scan to open the GitHub repository

AI Lawyer India provides general legal information only. It is not legal advice or a replacement for a qualified legal professional.